



Université de Sherbrooke

SQL

Types et domaines

SQL_02

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Ameni SOUID (ameni.Souid@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

—

Scriptorum/Scriptorum/SQL_02b-Types-et-domaines, version 1.0.1.a, en date du 2026-01-13

— document en cours de révision, ne pas citer à l'externe —

Sommaire

...

Mise en garde

Le présent document est en cours d'élaboration ; en conséquence, il est incomplet et peut contenir des erreurs.

Historique

diffusion	resp.	description
2026-01-13	CK	Récupération de notes diverses.

Table des matières

Introduction.....	4
1. Présentation.....	4
2. Définition de types de base	4
2.1. Syntaxe (PostgreSQL)	4
2.2. Exemples de types	4
2.3. Motivation.....	5
2.4. Transportabilité	5
3. Définition de sous-types.....	5
3.1. Syntaxe (PostgreSQL)	5
3.2. Exemples de sous-types.....	5
3.3. Motivation.....	6
3.4. Transportabilité	6
Conclusion.....	7
Références	8
Définitions	9

Introduction

Le présent document a pour but de présenter une synthèse des types qu'on retrouve dans SQL.

Dans un premier temps, nous présenterons une grammaire simplifiée, sous-ensemble propre de la grammaire de la norme ISO 9075:2016, avec, au besoin, certaines particularités de PostgreSQL.

1. Présentation

La théorie des types définit :

- type de base : la dénotation d'un ensemble fini de valeurs propres (non partagées avec un quelconque autre type de base);
- sous-type : la dénotation d'un sous-ensemble d'une type de base défini par une contrainte.

En SQL,

- le constructeur TYPE permet de définir un type de base ;
- le constructeur DOMAIN permet de définir un sous-type.

...

2. Définition de types de base

Le type de base est analogue au mécanisme de classe offert par des langages tels que Objective C, C++, C#, Java, etc.

2.1. Syntaxe (PostgreSQL)

Création des types composites

```
CREATE TYPE name AS  
  ( [ attribute_name data_type [, ... ] ] )
```

Création des types d'énumération

```
CREATE TYPE name AS ENUM  
  ( [ 'label' [, ... ] ] )
```

2.2. Exemples de types

```
create type Point_3D as  
  (x float, y float, z float);
```

```
create type Cote as enum  
  ('A', 'B', 'C', 'D', 'E');  
  
create type Saison as enum  
  ('Hiver', 'Printemps', 'Été', 'Automne');
```

2.3. Motivation

- Faciliter la définition et l'évolution de schémas.
- Encapsuler la définition de contraintes internes entre des attributs qui ne peuvent être interprétés indépendamment et qui, de ce fait, devrait déterminer un nouveau type les incorporant.

2.4. Transportabilité

Plusieurs constructeurs de types sont définis par le standard, mais ils sont rarement offerts par les dialectes SQL contemporains lorsqu'ils le sont, leur syntaxe et leur sémantique sont généralement différentes de celle décrite dans le standard ISO

3. Définition de sous-types

La définition d'une contrainte à un type de base.

La représentation des valeurs du domaine est celle du type de base.

3.1. Syntaxe (PostgreSQL)

```
CREATE DOMAIN name [ AS ] data_type
  [ DEFAULT expression ]
  [ domain_constraint [ ... ] ]

where domain_constraint is:

[CONSTRAINT constraint_name]
{ NOT NULL | NULL | CHECK (expression) }
```

3.2. Exemples de sous-types

```
create domain SigleCours
char(6) not null
constraint SigleCours_check check (
  value similar to '[A-Z]{3}[0-9]{3}'
  and
  value in {'IFT', 'IMN', 'IGE', 'GMQ'})
);
```

/!\ Attention

```
create domain Matricule
char(8) not null
constraint matricule_check check (
  value similar to '[0-9]{8}'
);
```

```
create domain Telephone
varchar(13)
constraint Telephone_check check (
  value similar to '[0-9]{8,13}'
);
```

```
create domain TauxEscompte
numeric(3,2)
constraint tauxEscompte_check check (
    value between 0.00 and 1.00
);
```

3.3. Motivation

- Assurer la cohérence d'un schéma en permettant la définition d'un type à partir d'un type de base et d'une contrainte.
- Faciliter la documentation, l'évolution et l'entretien des modèles logiques de données (particulièrement ceux de taille moyenne ou grande).

3.4. Transportabilité

Disponibilité variable selon le SGBD.

En cas d'absence, il faut alors utiliser CREATE TYPE dont la syntaxe et l'usage varient d'un dialecte à l'autre; la sémantique fait en sorte que les types ne peuvent pas toujours être substitués simplement aux domaines; la dénotation et les règles de compatibilité des valeurs diffèrent de celles des valeurs de domaine.

À moins de vouloir mettre en oeuvre un mécanisme uniforme et complet de vérification de la non-annulabilité, ne pas utiliser la contrainte NOT NULL par souci de transportabilité et d'homogénéité dans le traitement des types.

Conclusion

Références

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-09-16 16:46:06 UTC



Université de Sherbrooke